

게임 서버 프로그래머 Portfolio

MMORPG 서버 및 Jolt Physics 기반 서버 권위형 액션 RPG 제작 경험

Author: 박경준

Project Overview

Simplest MMORPG MINECRAFT

MMORPG Server

IOCP 기반 C++ 멀티스레드 서버. 동접 5000명 및 단일 맵 20만 객체 구동

Role: Server Architecture & Optimization

C++ 20

IOCP

DB

Lock-Free Queue



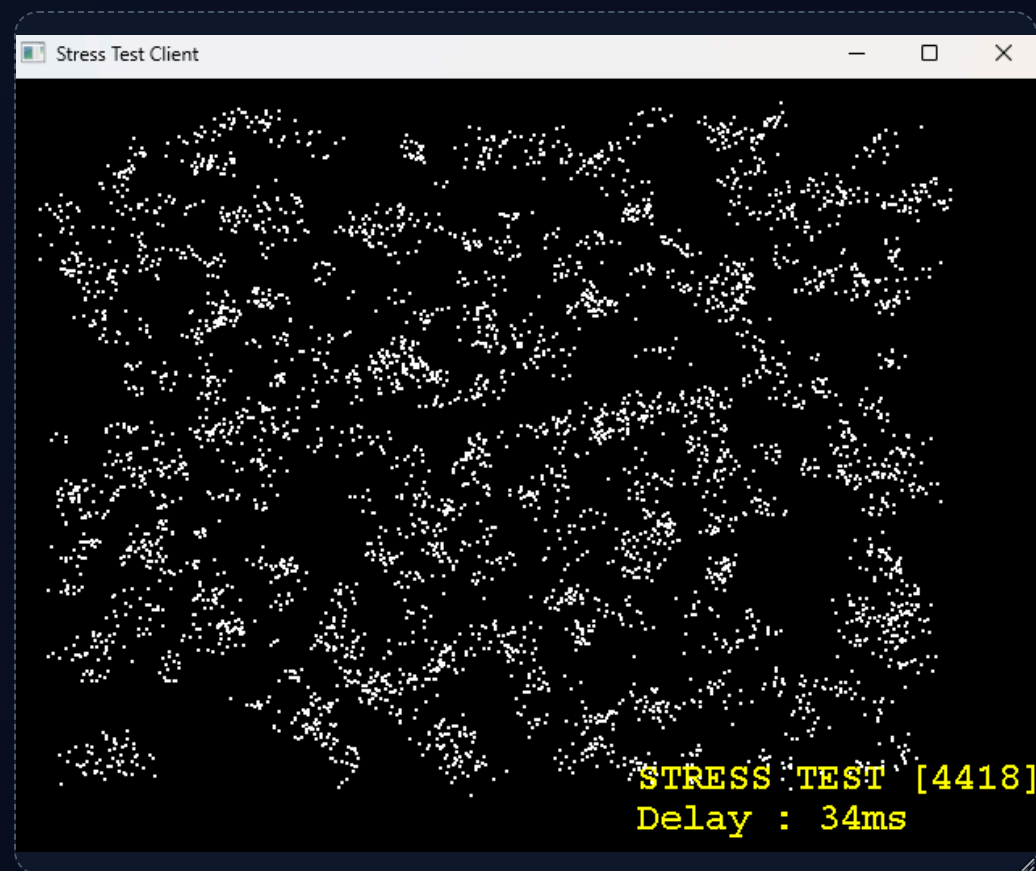
시야 처리 병목 최적화 (Sector Grid)

Problem: 20만 마리의 NPC와 유저가 서로의 거리를 $O(N^2)$ 로 계산하여 서버 톱 심각한 저하 발생

Solution: 공간을 고정 크기의 Grid(Sector)로 분할하고 `std::shared_mutex(Read-Write Lock)`를 적용하여 인접 섹터의 객체만 탐색하도록 변경

Performance Gain:

탐색 복잡도를 $O(K)$ ($1 < K < N$) 수준으로 낮추어 서버 딜레이 20~30ms 시 동시 접속자 수 4400명 안정적 유지 (별개 네트워크에 속한 컴퓨터의 더미 클라이언트 사용)



네트워크 I/O 병목 및 데이터 파편화 최적화

Problem: 매 틱마다 쏟아지는 수많은 객체의 이동/투사체 패킷 전송(Send) 시스템 콜 비용 과다

Solution: 전송 큐에 패킷을 버퍼링 후 WSABUF 배열을 이용한 Scatter-Gather I/O(Batching)로 뭉쳐서 보내고, 수신단에서 링 버퍼 기반 재조립(Reassembly) 파서 구현

Performance Gain:

WSASend 시스템 콜 횟수 대폭 감소 및 패킷 유실률 제로 달성

```
cpp
// 1. 시스템 콜 후 링버퍼에 데이터 수신 기록
cl->recv_buffer_.CommitWrite(num_bytes);

// 2. 링버퍼 기반 패킷 프래그멘테이션(Fragmentation) 안전 조립
while (true)
{
    unsigned char packet_size;
    // 패킷 헤더(크기) 1바이트 얻보기 (Wrap-around 안전 처리)
    if (!cl->recv_buffer_.Peek(reinterpret_cast<char*>(&packet_size), 1))
        break; // 데이터 부족

    // 완전한 패킷이 도착했는지 검증
    if (cl->recv_buffer_.GetStoredSize() < packet_size)
        break; // 포개진 패킷 대기 (Fragmentation 방어)

    // 안전하게 패킷 원본 복사 후 Head 전진 (Read)
    unsigned char packet_buf[256];
    cl->recv_buffer_.Read(reinterpret_cast<char*>(packet_buf), packet_size);

    // 완벽하게 조립된 패킷만 로직으로 전달
    cl->process_packet(packet_buf);
}
```

```
// 1. 송신 버퍼링 및 Scatter-Gather용 WSABUF 배열 세팅
EXP_OVER(std::vector<std::vector<char*>&& data>
    : type(io_type::send), pending_data(std::move(data)))
{
    ZeroMemory(&over, sizeof(over));
    wsa_bufs.reserve(pending_data.size());

    // 버퍼 복사 없이 포인터와 크기만 배열로 쓰기 (Scatter-Gather)
    for (auto& d : pending_data) {
        wsa_bufs.push_back({ static_cast<ULONG>(d.size()), d.data() });
    }
}

// 2. 비동기 락프리 기반 전송 (Flush)
void SESSION::flush_send() {
    std::vector<std::vector<char*>> sending_data;
    {
        std::lock_guard<std::mutex> lock(send_mtx);
        sending_data.swap(send_queue); // 큐에 쌓인 패킷을 한 번에 들고오기
    }

    EXP_OVER* send_over = new EXP_OVER(std::move(sending_data));
    DWORD send_bytes;

    // 모아둔 WSABUF 배열을 한 번의 시스템 콜로 일괄 전송
    WSASend(client_, send_over->wsa_bufs.data(),
        static_cast<DWORD>(send_over->wsa_bufs.size()),
        &send_bytes, 0, &send_over->over, NULL);
}
```

DB 스레드 분리 및 Stored Procedure 설계 및 구현

Problem: 메인 워커 스레드에서 ODBC(동기식)로 DB에 쿼리를 전송할 경우, 응답이 올 때까지 IOCP 스레드가 정지(Blocking)되어 전체 서버의 응답 불능(Starvation)을 초래함.

Solution: DB 전담 스레드 분리 & Lock-Free Queue: 유저의 저장/불러오기 요청은 큐에 밀어넣고 워커 스레드는 즉각 복귀하여 패킷 처리를 이어갑니다.

Stored Procedure 도입: 복잡한 인벤토리, 퀘스트, 스킬 데이터 저장을 하나의 DB 프로시저로 묶어 네트워크 왕복 비용을 최소화하는 것뿐만 아니라 보안적으로 DB의 구조를 알 수 없도록 도입하였습니다.

Performance Gain:

워커 스레드의 I/O 블로킹 되지 않음 풀부하 상황에서도 패킷 처리 지연(Lag) 방지가 가능해집니다.

```
void db_thread_func() {
    while (true) {
        DbJob job;
        if (db_queue.try_pop(job)) { // Lock-Free Queue 기반 작업 인계
            if (job.type == SAVE_PLAYER) {
                // Stored Procedure 쿼리 실행으로 통신 비용 최소화
                std::string query = "EXEC sp_save_player " + std::to_string(job.id);
                SQLExecDirect(hstmt, (SQLWCHAR*)query.c_str(), SQL_NTS);
            }
        }
    }
}
```

Lua Script를 활용한 확장성 있는 NPC AI

Problem: 20만 마리의 몬스터 패턴을 모두 C++로 하드코딩하기엔 한계가 있으며, 하나의 글로벌 lua_State를 사용하면 멀티스레드 환경에서 치명적인 Lock 병목이 발생합니다.

Solution: thread_local 기반 Lua VM 할당: 각 IOCP 워커 스레드마다 고유의 독립적인 lua_State 풀을 유지합니다.

서로 다른 스레드가 동시에 C++에서 Lua 스크립트를 호출해도 상호 배제(Lock)가 전혀 필요 없습니다.

Performance Gain:

기획자가 직접 AI 로직(lua)을 수정할 수 있는 극대화된 확장성 확보 및 재컴파일 및 유지보수가 용이합니다

```
// npc_ai_manager.h 의 핵심 로직
static lua_State* get_state(int npc_type) {
    // thread_local 맵을 사용하여 스레드별로 안전하게 분리된 Lua s
    thread_local std::unordered_map<int, lua_State*> pool;

    if (pool.find(npc_type) == pool.end()) {
        lua_State* L = luaL_newstate();
        luaL_openlibs(L);
        luaL_dofile(L, get_script_path(npc_type).c_str()); //
        pool[npc_type] = L;
    }
    return pool[npc_type];
}
```

Project Overview

PIP Project

Action RPG Game S.T.L (DirectX 12 + Jolt Physics)

프레임 단위 정밀 타격을 위한 서버 권위형 액션 RPG

Role: Server Physics & AI Sync

C++

Jolt Physics

Behavior Tree

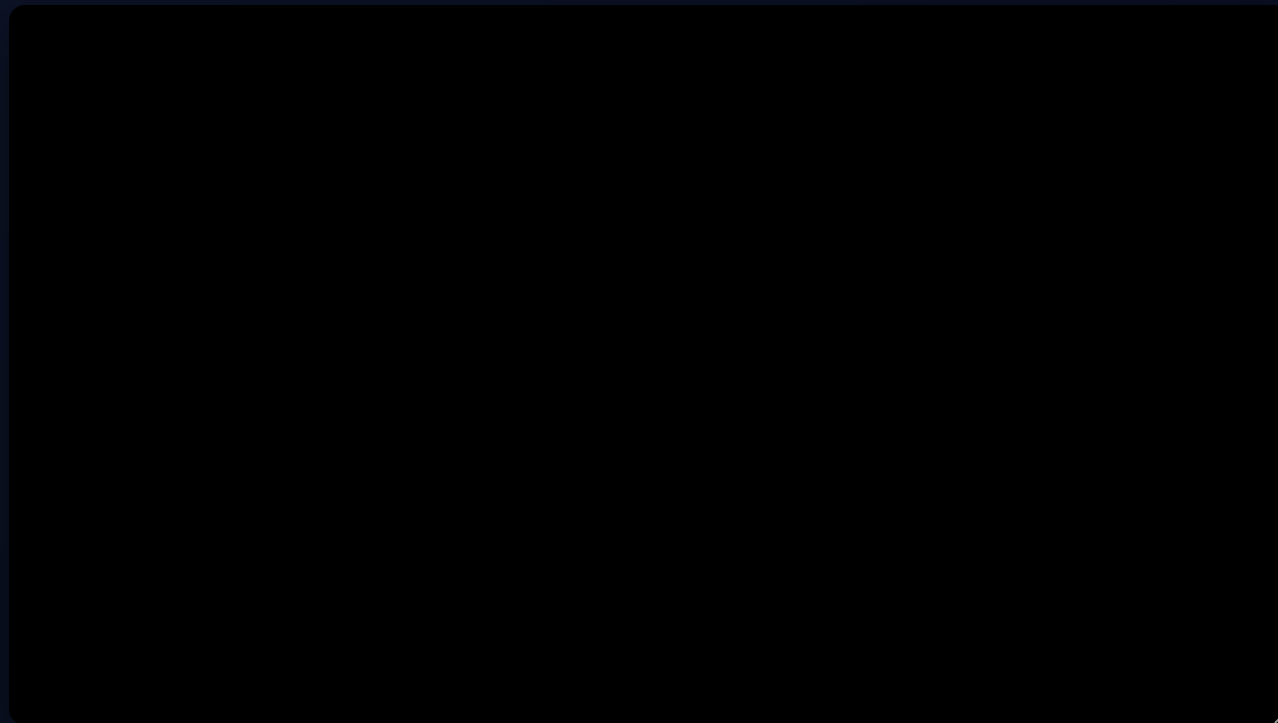


조 작 법

이동	W, A, S, D
대쉬 / 회피	Left Shift
점프	Space Bar
상호작용	F
기본 공격	마우스 좌클릭
스킬 사용	마우스 우클릭
락온 토글	마우스 휠 클릭 또는 Tab
퀘스트 및 스토리 확인	Q
조작법 확인	E

Game Demo

게임 데모 영상입니다.



영상 바로가기 🖱️

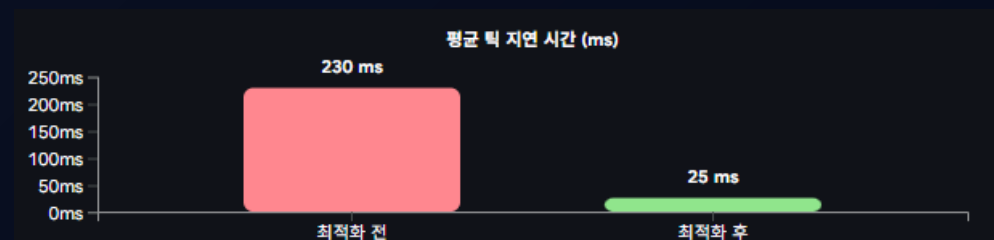
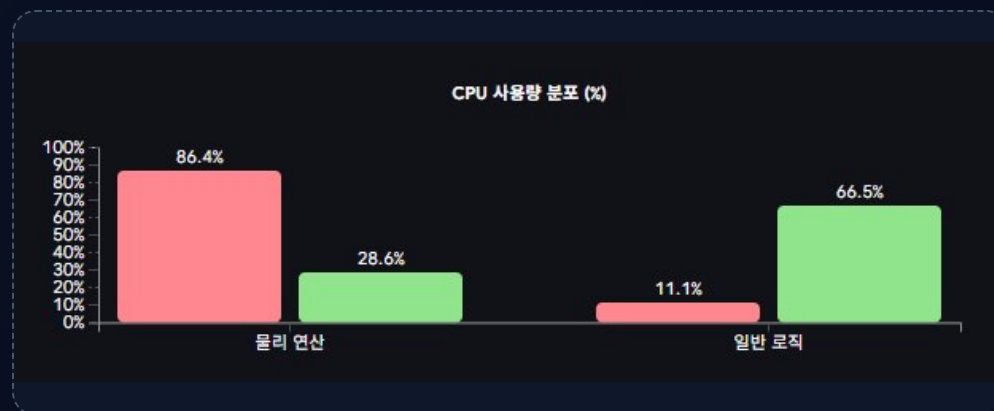
서버 물리 엔진(Jolt) 부하 억제 설계

Problem: 다수의 플레이어와 몬스터 객체 물리 연산을 서버가 모두 담당할 경우 심각한 물리 스레드 병목 발생

Solution: 대기 중인 NPC의 Motion Type을 Kinematic으로 설정 시야 처리를 통해 멀리있는 NPC의 물리 업데이트를 해제하여 연산에서 제외하고, 근접 시 Dynamic으로 자동 승격(Dynamic Promotion)과 물리 엔진 업데이트 -> ray cast방식으로 경량화

Performance Gain:

충돌 연산 부하 최소화를 통한 안정적 서버 물리 프레임 타임 확보
50000마리 NPC + 400명 플레이어 부하 테스트시 안정적인 부하 밸런싱



정밀 피격 동기화를 위한 지연 보상 (Rewind)

Problem: 네트워크 지연(Ping)으로 인해 클라이언트 화면의 몬스터 위치와 서버 위치가 달라 타격감 훼손

Solution: 모든 물리 객체의 최근 1초간 위치/히트박스 스냅샷을 원형 큐에 저장. 공격 패킷 타임스탬프 기반 과거 시간으로 Rewind 후 판정 처리

Performance Gain:

Ping 100ms 환경에서도 불일치 없는 완벽한 서버 권위형 피격 판정 달성



Behavior Tree(BT)를 활용한 보스 AI

Problem: 보스의 복잡한 페이즈 변화, 스킬 연계, 어그로 타겟 상실 등 다양한 예외 상황을 기존의 FSM(상태머신)으로 짜면 스파게티 코드가 되어 유령 포인터(Dangling Pointer) 및 서버 크래시를 유발함.

Solution: BT 모듈화 아키텍처: std::shared_ptr<BTNode> 기반의 트리 구조로 AI를 설계해 조건(Decorator)과 행동(Action)을 엄격히 분리했습니다.

빌더 패턴을 도입하여 트리 구조로 노드들을 붙일 수 있으며 블랙보드로 노드들의 공용데이터를 공유하도록 설계함

Performance Gain:

현재 상태 체크 시간 획기적 단축, 재사용 가능한 AI 노드 라이브러리 확보

```
BTBuilder builder;
auto root = builder
    .sequence()
    .leaf<Condition_IsAlive>()
    .leaf<Condition_IsHitted>(DecoratorType::Inverter)
    .selector() // 전체 로직 우선순위 결정
    { // --- [우선순위 1] 전투 로직 ---
        .sequence()
        .leaf<Condition_HasEnemy>() // 타겟(적군)이 있는가?
        .leaf<Condition_IsTargetInLeashRange>(25.0f) // 25m 내에 있으면 추격 유지
        .selector()
        // 1-1. 강력한 공격 시도 (사거리 2.5m)
        .sequence()
        .leaf<Condition_IsEnemyInRange>(1.5f)
        .leaf<Action_AttackEnemy>(heavyAtk)
        .end()
        // 1-2. 일반 공격 시도 (사거리 1.5m)
        .sequence()
        .leaf<Condition_IsEnemyInRange>(1.5f)
        .leaf<Action_AttackEnemy>(normalAtk)
        .end()
        // 1-3. 타겟 사거리 밖이면 네비메쉬 추격
        .sequence()
        .leaf<Action_UpdateEnemyPosToTarget>()
        .leaf<Action_FindPath>("MainStage_NavMesh")
        .leaf<Action_FollowPath>(6.0f) // 추격 속도 6.0
    }
```

Data Race가 없는 안전한 Room 아키텍처

Problem: 같은 인스턴스(Room)에 속한 몬스터와 유저들의 상태(HP, 위치)를 여러 워커 스레드가 동시에 수정하려 들면, 복잡한 Deadlock(교착상태)과 Data Race(데이터 오염)가 필연적으로 발생하고 이는 MO게임 기획 단계에서 없앨 수 있었습니다.

Solution: Job Queue 기반 로직 직렬화: 방(Room)마다 고유한 Command Queue를 부여하여 상태를 변경하는 모든 이벤트(공격, 이동, 버프)를 람다(Lambda)로 캡처해 Push 합니다.

한 번에 단 하나의 스레드만 해당 룸의 Job 큐를 비워냄으로써 로직이 순차적으로 실행됩니다.

Performance Gain:

객체별 Mutex Lock 오버헤드 완전 제거로 CPU 사이클을 절약하고, 데드락 구조를 원천 차단하여 초고속 전투 연산 달성 -> 물리엔진 루프에 알맞는 구조를 부가적으로 얻을수 있었습니다

```
void PacketHandler::handle_attack(Session* session, AttackPacket* packet)
{
    auto room = session->get_room();
    if (!room) return;
    // Data Race를 막기 위해 해당 룸의 전용 JobQueue로 로직을 밀어넣음
    room->PushJob([room, session, packet]() {
        // 이 블록 안의 코드는 오직 하나의 스레드에서만 안전하게 실행 보장!
        auto target = room->find_monster(packet->target_id);
        if (target) {
            target->take_damage(packet->damage);
        }
    });
}
```

기술 성과 2026 한국게임학회 춘계 학술발표대회 발표

성과: 포스터 발표 부문으로 참가하여 기술적 성과를 발표 하였습니다

PIP 2026 한국게임학회 춘계 학술대회 발표 논문 PDF 다운로드

**한국공학대학교**
TECH UNIVERSITY OF KOREA

2026 한국게임학회 춘계 학술발표대회

**한국게임학회**
Korea Game Society

3D MO 액션 게임을 위한 gITF 2.0 표준 파서 및 스킨링 애니메이션 구현과 PBR 파이프라인 및 오픈소스 물리엔진 서버 최적화 연구

한국공학대학교 박대원, 임찬진, 박경준, 정내훈

Introduction

외부 라이브러리 없는 독자적 gITF 2.0 파서 개발 및 스킨링 애니메이션 시스템 구현
DirectX 12 기반 실시간 PBR 파이프라인 구축으로 시각적 사실성 및 렌더링 성능 확보
IOCP 서버와 Jolt Physics 엔진 연동을 통한 3D MO 게임 맞춤형 물리 시뮬레이션 환경 완성
스냅샷 리와인드 지연 보상 및 물리 LOD 적용으로 다중 접속 환경의 물리 연산 병목 해소

gITF 2.0 파싱 및 스킨링 애니메이션 시스템

- 외부 라이브러리 없이 바이너리를 직접 1차원 배열로 파싱해 캐시 효율을 높이고, 스케일 행렬을 곱해 원소 좌표계 이기종 문제를 해결
- 이진 탐색을 적용해 키프레임 탐색 시간을 단축하고, 마스터 스케일러트 기반의 소켓 시스럼으로 무기 장치를 구현
- 선형 할당기를 설계하여 렌더링 시 발생하는 할당 오버헤드와 단편화를 차단
- 마스터 OBB 기반의 계층적 BVH 구조를 통해, 메쉬와 카메라 간의 충돌 연산 부하를 대폭 감소



Jolt Physics 서버 연동 및 물리 엔진 최적화

- Room 인스턴스 단위로 독립적인 물리 공간을 격리하고, 계층적 충돌 필더링을 통해 Broad-phase 단계의 불필요한 충돌 창을 차단함
- 정적 맵 데이터(StaticCompoundShape)를 단일 생성 후 인스턴싱하여 메모리를 최적화함
- 거리 기반 물리 LOD를 적용하여, 원거리 NPC는 RayCast 제어로 전환하고 매인 루트 호출을 제한함



- Transform, CharacterController, NPCController 등을 계층적 컴포넌트 구조로 분리하여 로직 제어 효율성을 높임
- 부위별 Hitbox 데이터와 스냅샷 리와인드(Snapshot Rewind)를 결합하여, 공격 패킷 수신 시 과거 위치를 역산해 지연을 보상함

실시간 PBR 파이프라인 및 IBL 통합

- ORM 텍스처를 분리하여 차폐(e), 거칠기(G), 금속성(B) 파라미터로 해석해 메타 대역폭을 절약
- Cook-Torrance BRDF 모델을 적용하여 직접광 반사를 구현
- Diffuse IBL은 3개의 SH 계수를 통해 연산량을 감축
- Specular IBL은 Split-Sum Approximation을 도입해 LUT를 조합하여 실시간 렌더링
- Fresnel 근사식을 적용하여 금속/비금속 재질의 물리적 반사 특성 구현
- 표준 선형 오일 기반의 시각적 검증용 PBR 파이프라인의 렌더링 신뢰성 입증



Result

- 독자적 gITF 파서로 메모리 캐시 효율을 극대화하고 파싱을 보장, 이진 탐색을 통해 애니메이션 연산을 O(log N)으로 단축하고 동시에 마스터 스케일러트 소켓 시스템을 구현하였고, 선형 할당기 제작을 통해 렌더링 시의 메모리 단편화를 차단하였으며, 마스터 OBB 기반 계층적 BVH를 도입해 실시간 충돌 연산 부하를 대폭 감소
- Cook-Torrance BRDF와 향상된 IBL 기반(SH 계수, Split-Sum 근사)을 적용한 최적화된 PBR 파이프라인을 구축하고 gITF 2.0 파서를 결합해 연산 부하를 절감함으로써, 상용 엔진 없이도 수준의 시각적 사실성과 실시간 렌더링 성능을 입증
- 서버 권위적 안정, 스냅샷 기반 지연 보상 시스템을 결합하여, 클라이언트 번조를 원만 차단하고 네트워크 지연 상황에서도 높은 다중 접속성을 보장 가능하며, 400명 플레이어 + 5만마리 NPC 동시 물리시뮬레이션 가능한 아키텍처 구현

Thank You!

서버 프로그래머 포트폴리오를 검토해 주셔서 감사합니다.

GitHub: <https://github.com/AACHANJINAA/PIP>

GitHub: <https://github.com/pkj7323/ServerGP>

Contact: pkj73231015@gmail.com

